

Tiết 6/9

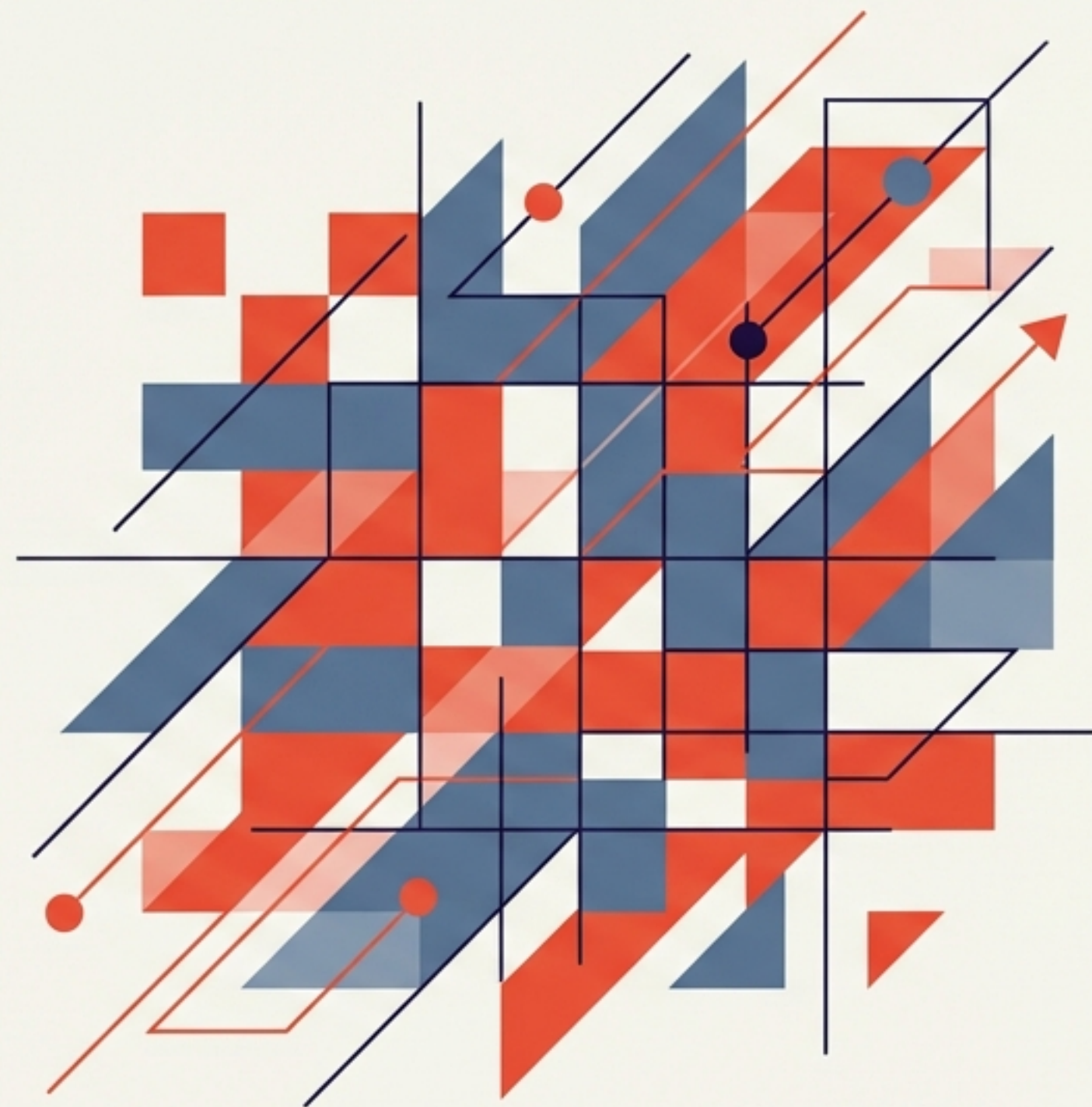
CHƯƠNG 4: PHÁT HIỆN VÀ MÔ TẢ ĐẶC TRƯNG ẢNH

Buổi 2 - Tiết 2: Thuật toán SURF (Speeded Up Robust Features)

Môn học: Thị giác máy tính (INF03111)

Giảng viên: Trần Thành Thắng

Lớp: IT23M



Mục tiêu bài học



Kiến thức

- Hiểu lý do SURF ra đời: Đáp ứng nhu cầu tối ưu tốc độ và xử lý thời gian thực.
- Nắm vững cơ chế Ảnh tích phân (Integral Images): Chìa khóa vàng của SURF.
- Giải thích vai trò của Bộ lọc Box (Box Filters): Xấp xỉ đạo hàm toán học.

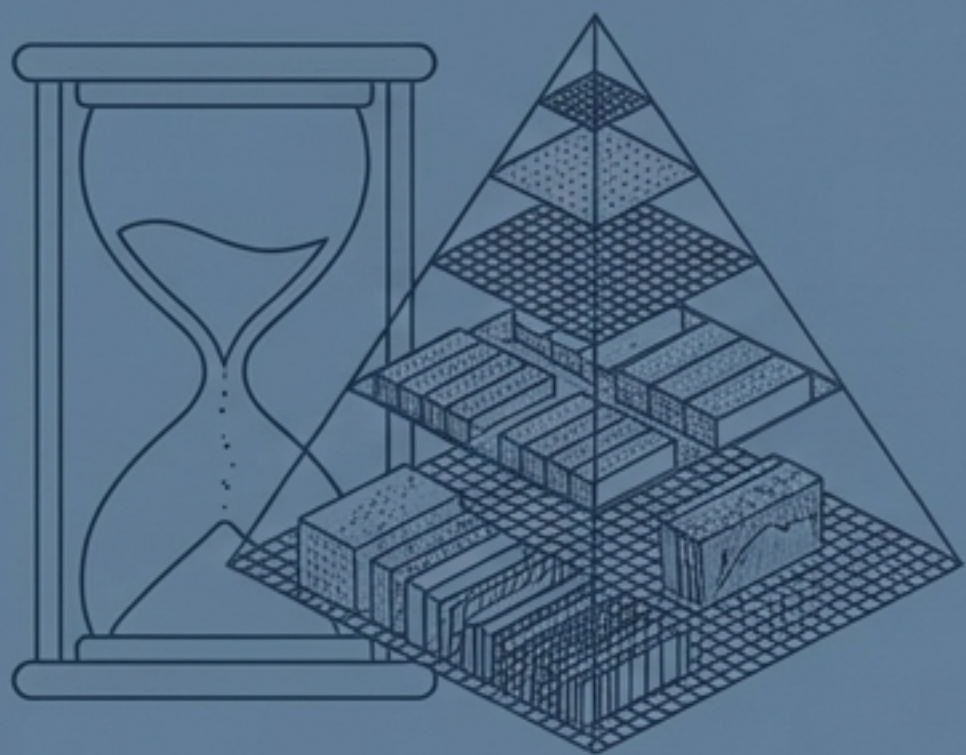


Kỹ năng [CLO1 & CLO4]

- Phân tích và đối chiếu: Đánh giá ưu/nhược điểm giữa SIFT và SURF.
- Ra quyết định: Lựa chọn thuật toán phù hợp cho từng bài toán thực tế.

Vấn đề cốt lõi: Bài toán Tốc độ

SIFT: Tiêu chuẩn vàng nhưng chậm chạp



SIFT cung cấp độ chính xác tuyệt đối nhưng quy trình xây dựng kim tự tháp Gaussian quá nặng nề, không thể đáp ứng...

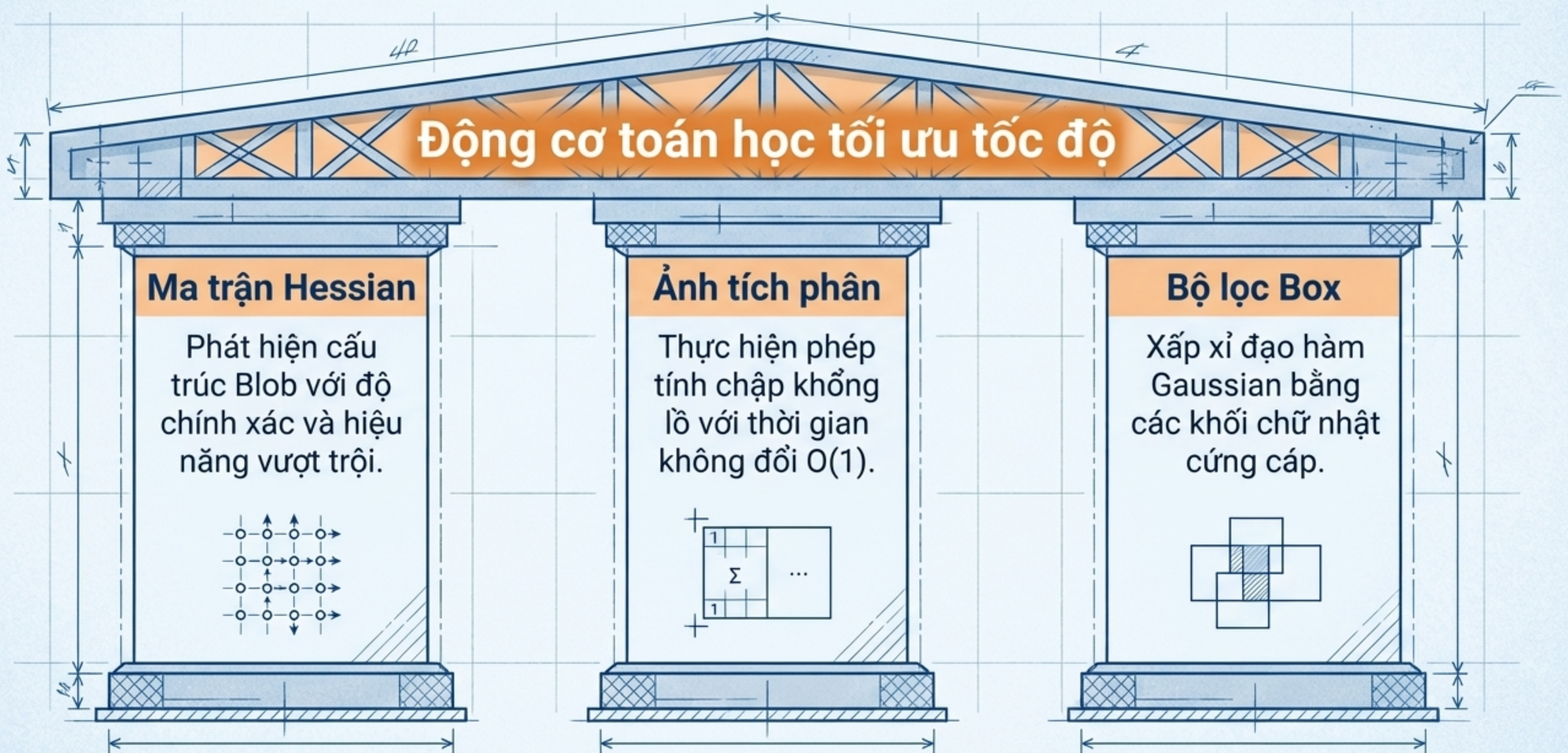
Thực tế: Nhu cầu thời gian thực



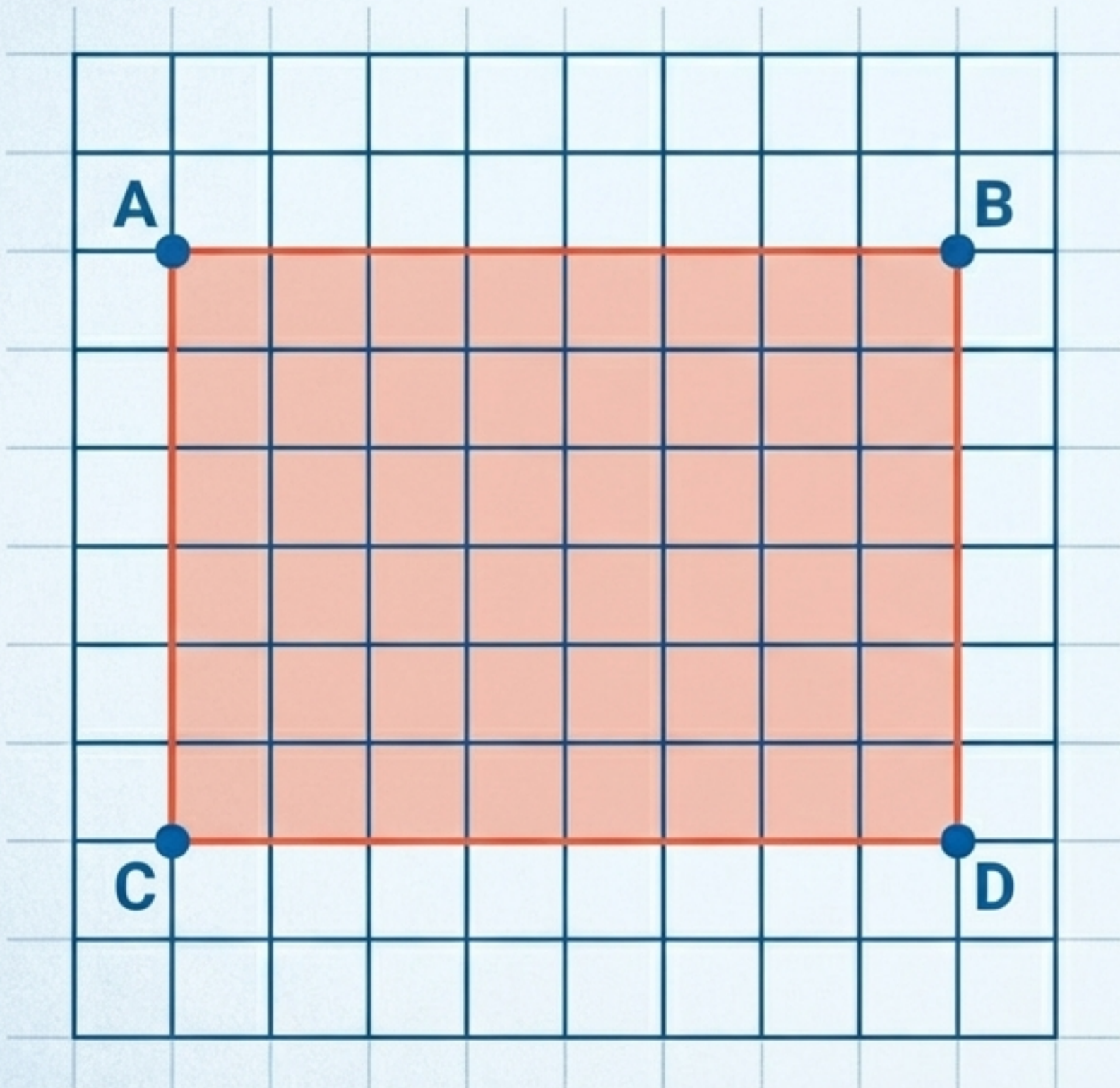
...AR, Xe tự lái và Robot không thể chờ đợi. Chúng yêu cầu tốc độ phản hồi tính bằng mili-giây.

Giải pháp (2006): Thuật toán SURF bẻ khóa giới hạn tốc độ nhưng vẫn giữ nguyên độ mạnh mẽ.

Cấu trúc DNA của thuật toán SURF



Vũ khí bí mật #1: Ảnh tích phân (Integral Images)



Tại pixel (x,y) , giá trị ảnh tích phân = Tổng toàn bộ pixel nằm bên trên và bên trái nó.

Sức mạnh bề khóa $O(1)$

Tính tổng cường độ sáng của một vùng chữ nhật bất kỳ chỉ tốn đúng:

- ✓ - 4 phép truy xuất bộ nhớ
- ✓ - 3 phép toán cộng/trừ

Kích thước bộ lọc lớn đến đâu, tốc độ vẫn không đổi!

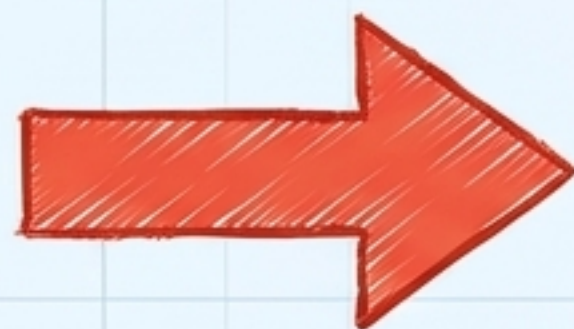
Vũ khí bí mật #2: Ma trận Hessian

$$H(x, y) = \begin{bmatrix} \frac{\partial^2 D(x, y)}{\partial x^2} & \frac{\partial^2 D(x, y)}{\partial x \partial y} \\ \frac{\partial^2 D(x, y)}{\partial y \partial x} & \frac{\partial^2 D(x, y)}{\partial y^2} \end{bmatrix}$$



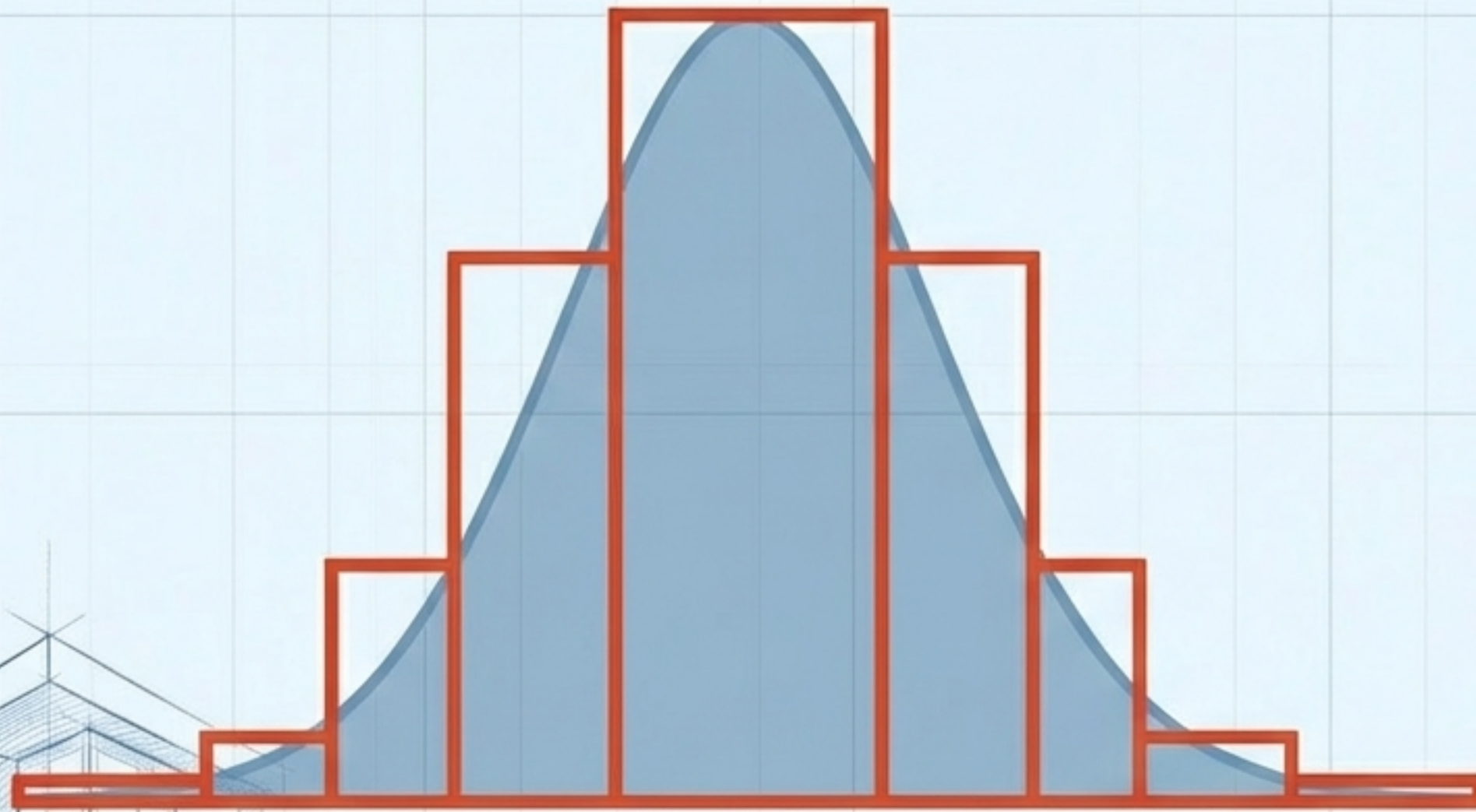
Tính Định thức
(Determinant) của
ma trận.

Nhiệm vụ: Định vị các
điểm cấu trúc dạng
đốm (blobs) siêu tốc.



Định thức đạt cực
đại → Ứng viên
Keypoint tiềm năng.

Vũ khí bí mật #3: Bộ lọc Box (Box Filters)



Vấn đề của SIFT:

Tích chập đạo hàm Gaussian cực kỳ tốn tài nguyên.

Giải pháp của SURF:

“Bao vuông” Gaussian!

Dùng các khối hình chữ nhật để xấp xỉ đạo hàm.

Tương thích 100% với Ảnh tích phân → **Tính toán ngay lập tức!**

Công thức định thức siêu tốc

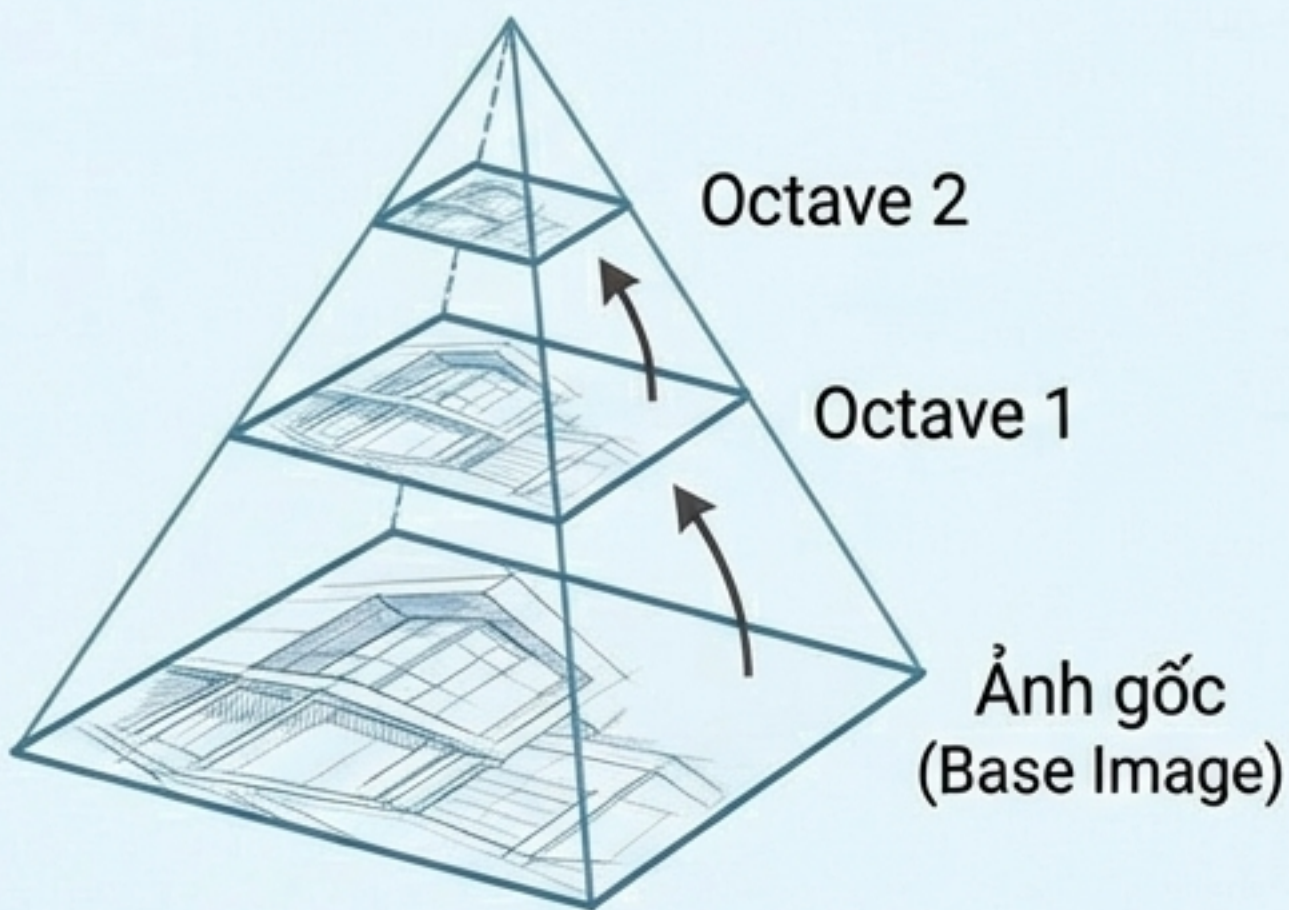
Bộ lọc Box + Ảnh tích phân = Xấp xỉ đạo hàm không suy giảm tốc độ.

$$\text{Det}(\mathcal{H}_{\text{approx}}) = D_{xx}D_{yy} - \mathbf{0.9}D_{xy})^2$$

Hệ số hiệu chỉnh: Bù đắp sai số giữa bộ lọc **Box vuông vức** và phân phối Gaussian thực tế.

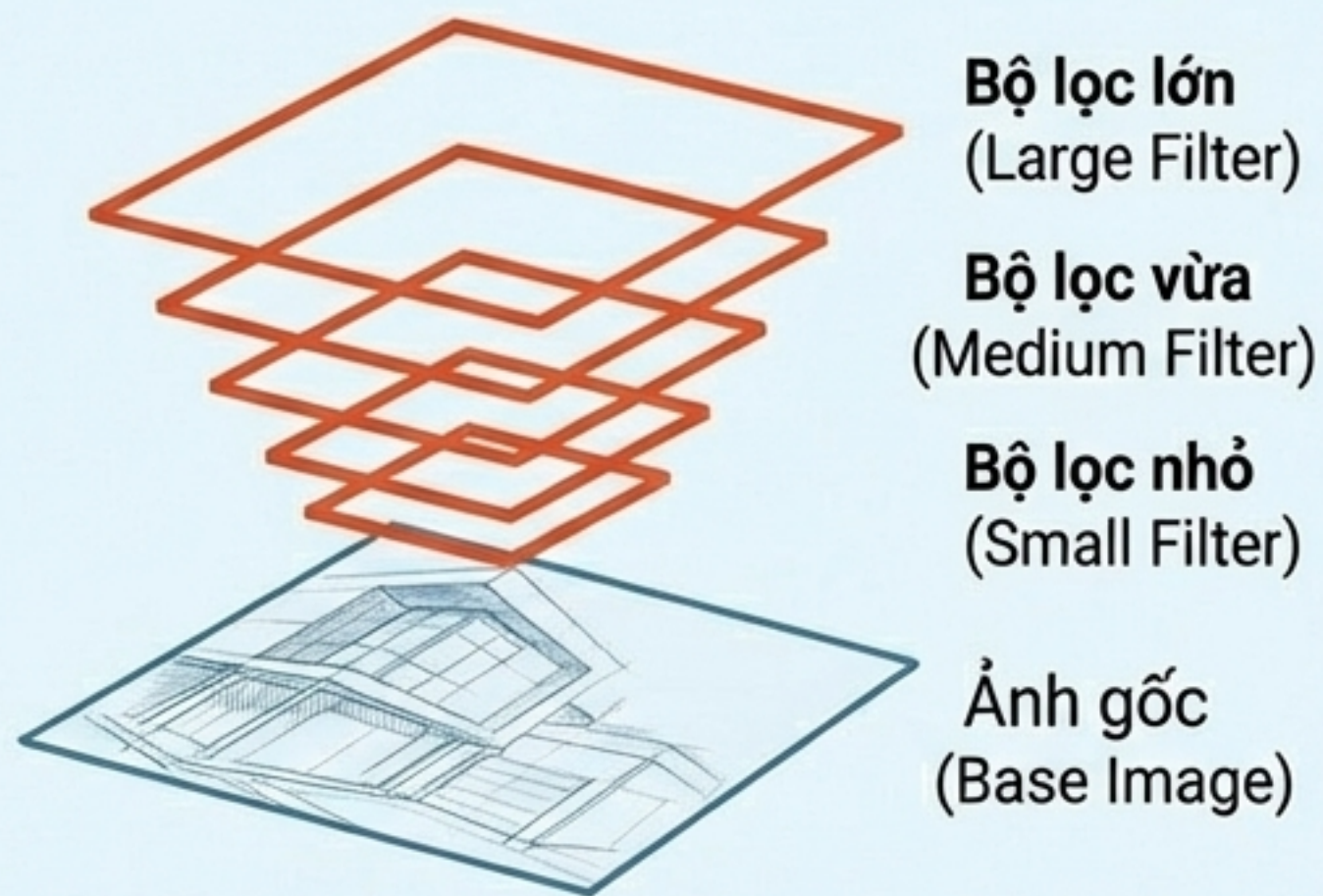
Không gian Tỷ lệ (Scale Space) kiểu mới

Thu nhỏ ảnh (Downsampling)



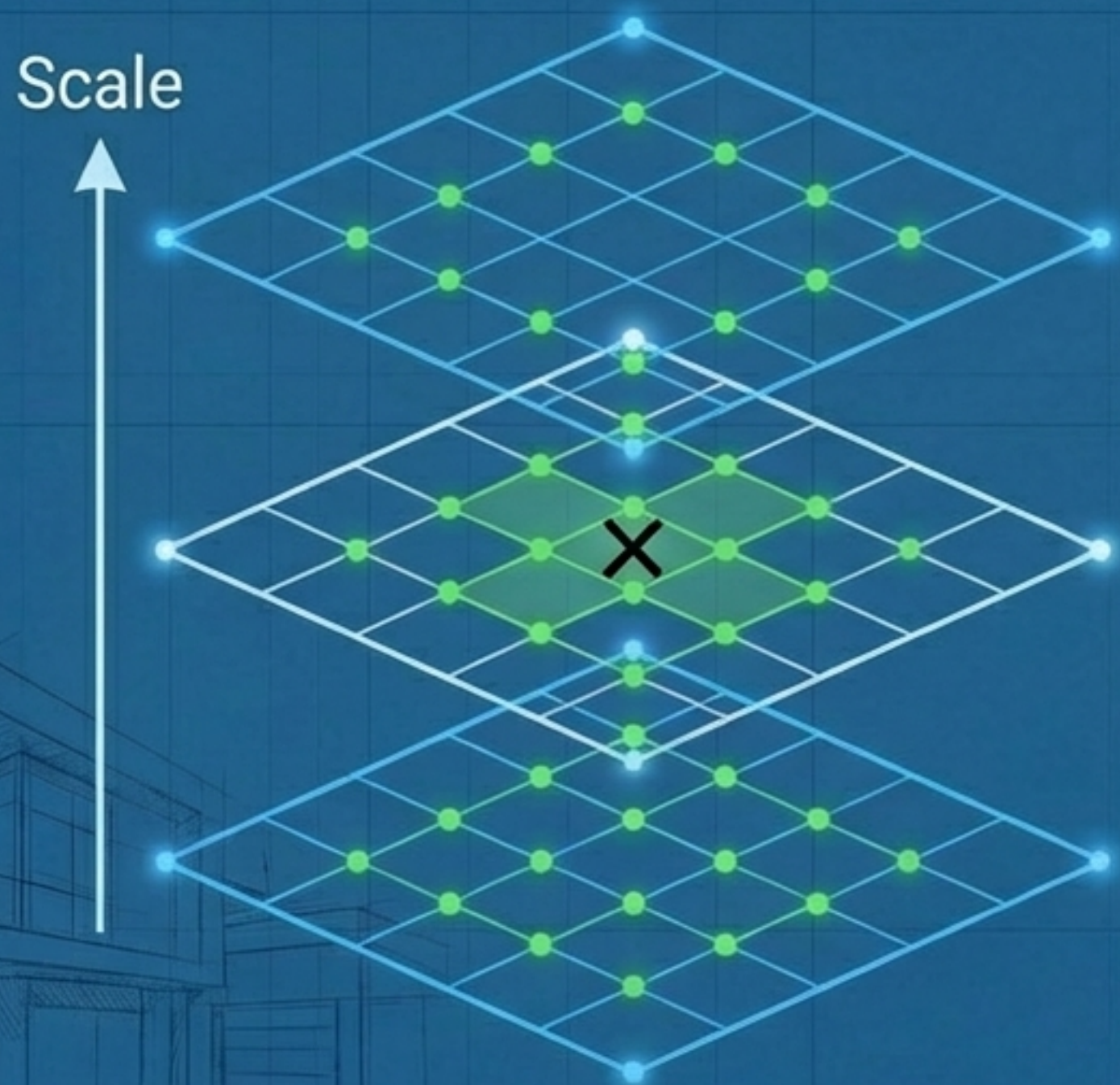
Giảm kích thước ảnh → Dễ mất thông tin, xử lý tuần tự chậm chạp.

Phóng to bộ lọc (Upscaling)



Giữ nguyên ảnh gốc, chỉ phóng to bộ lọc Box. Không mất điểm ảnh gốc & Chạy song song cực kỳ hiệu quả.

Định vị Cực trị & Ngưỡng Hessian



1. **Ước chế phi cực đại (NMS):** So sánh định thức Hessian của 1 pixel với 26 lân cận (8 cùng tầng, 9 tầng trên, 9 tầng dưới).
2. **Bộ lọc Hessian Threshold:** Chỉ giữ lại các điểm có định thức vượt qua ngưỡng an toàn.
3. **Định vị tinh tế (Sub-pixel):** Toán học nội suy tọa độ chính xác dưới **mức pixel**, triệt tiêu sai số cục bộ.

Giai đoạn 3: Gán hướng chính (Bất biến phép xoay)



Công cụ cốt lõi:

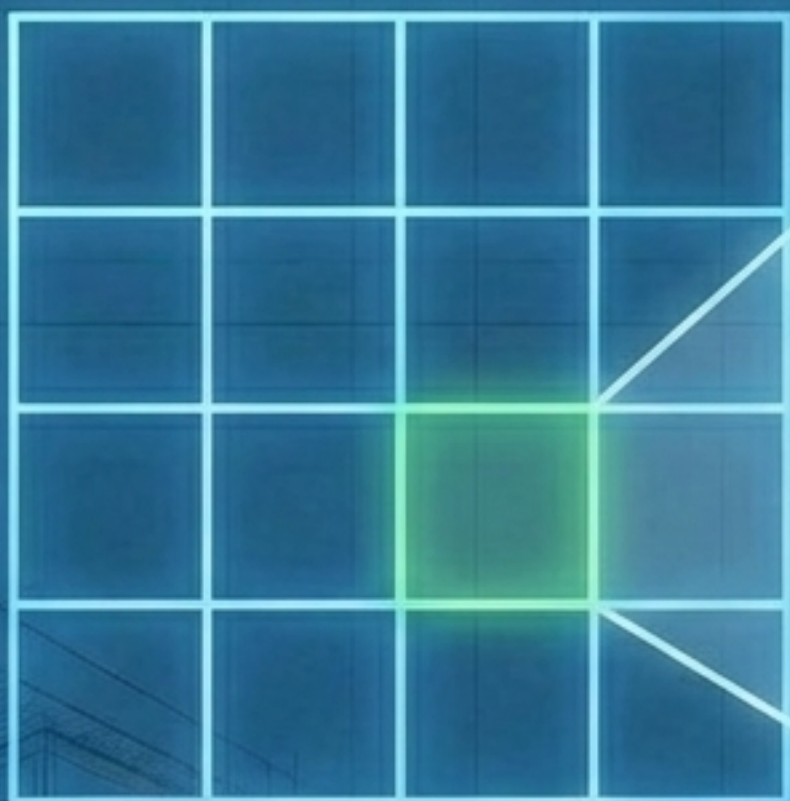
- Phản hồi Haar-wavelet theo hướng X và Y (tính siêu tốc bằng Ảnh tích phân).

Cơ chế Radar:

- Một cửa sổ 60 độ quét xoay vòng quanh vùng lân cận 6σ của keypoint.

Ấn định kết quả: Hướng có tổng vectơ phản hồi lớn nhất trở thành “Hướng chính” của đặc trưng.

Giai đoạn 4: Tạo Vector Đặc trưng 64 chiều



1. $\sum d_x$

2. $\sum d_y$

3. $\sum |d_x|$

4. $\sum |d_y|$

Chia lưới không gian: Vùng ảnh quanh keypoint được chia thành lưới 4x4 ô.

Trích xuất siêu nén: 16 ô × 4 giá trị phản hồi = Vector 64 chiều.

Gọn nhẹ hơn một nửa so với 128 chiều của SIFT

→ Tối ưu tuyệt đối cho đối sánh thời gian thực.

Trận chiến vương quyền: SIFT vs. SURF

Tiêu chí	SIFT (David Lowe)	SURF (Bay et al.)
Tốc độ	Chậm (Tích chập Gaussian)	Nhanh gấp 3-10 lần (Ảnh tích phân)
Cơ chế lõi	DoG (Hiệu của Gaussian)	Ma trận Hessian & Box Filter
Không gian tỷ lệ	Thu nhỏ ảnh gốc	Phóng to bộ lọc Box
Chiều dài Vector	128 chiều	64 chiều (Tối ưu bộ nhớ)
Độ bất biến	Tỉ lệ, Xoay, Chiếu sáng	Tỉ lệ, Xoay, Chiếu sáng

Thực chiến: Lập trình SURF với OpenCV

```
# Khởi tạo thuật toán SURF với ngưỡng Hessian  
surf = cv2.xfeatures2d.SURF_create(hessianThreshold=400)  
  
# Phát hiện Keypoint và trích xuất Vector mô tả 64 chiều  
kp, des = surf.detectAndCompute(img, None)
```

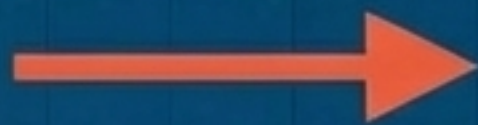
Mẹo thực chiến: Giá trị *hessianThreshold* càng cao
→ Số lượng điểm thu được càng ít, nhưng độ ổn định và tin cậy càng lớn.

Tổng kết **Tiết 6** & Hành trình tiếp theo

Cốt lõi của SURF

1. Nhanh hơn nhờ **Ảnh tích phân** & **Bộ lọc Box**.
2. Định vị bằng **Hessian**, mô tả bằng **Haar-wavelet**.
3. Vector 64 chiều tối ưu thời gian thực.

Vượt ra ngoài các Điểm đặc trưng



SIFT & SURF tìm các “**Điểm**”. Nhưng nếu ta muốn máy tính hiểu toàn bộ “**Hình dáng**” (như một người đang đi bộ) thì sao? Khám phá **Đặc trưng vùng HOG** và **Kết cấu LBP** ở tiết học sau.